

INTERVIEW PREP

OAuth vs JWT

What they are, how they work together, and how to explain the difference without falling into interview traps.

2026 edition

Built for frontend, backend, full-stack, platform, security, and system-design interviews.

THE ONE-LINE MENTAL MODEL

OAuth is an authorization framework. JWT is a token format. OAuth may issue a JWT access token, but OAuth does not require JWT, and JWT can be used outside OAuth.

If you only remember three things:

- OAuth answers how an application gets delegated permission to access a resource.
- JWT is a compact claims container that can be signed and/or encrypted.
- Authentication is not OAuth by itself. OpenID Connect (OIDC) adds an identity layer on top of OAuth 2.0.

Standards basis: RFC 6749, RFC 7519, RFC 8725, RFC 9068, RFC 9700, RFC 10017, plus the OAuth 2.1 working draft status as of Sep 2026.

01 The interview-ready answer

Use this first. Then go deeper only if the interviewer follows up.

“OAuth is about delegated authorization. JWT is a format for representing claims in a token.”

A strong 30-second answer

SAY THIS

OAuth and JWT solve different problems. OAuth is a framework for delegated authorization: it lets a client application obtain limited access to a protected resource without asking the user to hand over their password. JWT, or JSON Web Token, is a compact token format for carrying claims such as issuer, subject, audience, roles, scopes, and expiry. An OAuth access token can be a JWT, but it can also be an opaque string. JWTs can also be used outside OAuth. And if the question is about logging a user in, I would distinguish OAuth from OpenID Connect, which adds authentication and ID tokens on top of OAuth 2.0.

Your original version, tightened for interviews

- OAuth: “Which client is allowed to do what, on whose behalf, and with what scope?”
- JWT: “How claims are packaged into a portable token representation.”
- Relationship: “OAuth can use JWTs, but it does not depend on them.”

Avoid these answers

Weak answer	Better correction
“OAuth is authentication.”	OAuth is primarily authorization. OIDC is commonly used for authentication.
“JWT is an auth protocol.”	JWT is a token format / claims representation, not an authorization protocol.
“OAuth tokens are JWTs.”	They may be JWTs, but the access-token format is not mandated by OAuth 2.0.
“JWT is encrypted.”	Not necessarily. Signed JWT payloads are readable unless the JWT is encrypted as JWE.

02 OAuth: what the interviewer expects you to know

Think of OAuth as a permission handoff, not a token shape.

The four core roles

Resource owner	Usually the user who owns or controls the data.
Client	The app requesting access.
Authorization server	Authenticates/authorizes the interaction and issues tokens.
Resource server	The API that accepts the access token and serves protected resources.

Modern authorization-code flow, simplified

1	Client sends the user to the authorization server with requested scopes and a PKCE challenge.
2	User signs in / approves access at the authorization server.
3	Authorization server redirects back with a short-lived authorization code.
4	Client exchanges the code plus PKCE verifier at the token endpoint.
5	Authorization server returns an access token, and optionally a refresh token.
6	Client sends the access token to the resource server API.

2026 SECURITY SIGNAL

For redirect-based flows, be ready to mention Authorization Code + PKCE. RFC 9700 requires PKCE support, requires public clients to use it, recommends it for confidential clients, and says clients generally should not use the implicit grant. The resource-owner password credentials grant must not be used.

Scopes

A scope is a string representing requested or granted permissions, for example read:photos or calendar.read. The resource server still has to enforce authorization. A token saying “read” does not magically bypass your application’s access-control rules.

03 JWT: anatomy, claims, and validation

JWT stands for JSON Web Token. It is a compact, URL-safe way to carry claims.

The familiar three-part signed JWT

HEADER . PAYLOAD . SIGNATURE

A common signed JWT (JWS) is three Base64url-encoded segments separated by dots. Base64url is encoding, not encryption.

Illustrative decoded content

HEADER

```
{
  "alg": "RS256",
  "typ": "at+jwt"
}
```

PAYLOAD / CLAIMS

```
{
  "iss": "https://auth.example.com",
  "sub": "user_123",
  "aud": "https://api.example.com",
  "exp": 1788700000,
  "scope": "profile.read orders.read"
}
```

Claims worth knowing

Claim	Meaning
iss	Issuer: who issued the JWT.
sub	Subject: who/what the claims are about.
aud	Audience: which recipient(s) the token is intended for.
exp	Expiration time.
nbf	Not before: token should not be accepted before this time.
iat	Issued at time.
jti	Unique token identifier, often useful for replay/revocation strategies.

Signing vs encryption

- JWS: protects integrity/authenticity. The payload is still readable.
- JWE: encrypts the content for confidentiality.

04 How OAuth and JWT fit together

This is the part that separates “I memorized the definitions” from “I understand the system.”

OAuth does not prescribe one access-token format

TWO VALID OAUTH DESIGNS

Opaque access token: the API may introspect it or look up server-side state.

JWT access token: the API can often validate the signed token locally using the issuer’s keys and claims.

Token taxonomy

Token	Primary consumer	Typical purpose	JWT?
Access token	Resource server / API	Authorize API access	May be JWT or opaque
Refresh token	Authorization server	Obtain new access tokens	Usually treated as opaque by client
ID token (OIDC)	Client application	Tell client about authenticated user/session	JWT by OIDC design
Authorization code	Authorization server token endpoint	Temporary credential in code flow	Not an API access token

Classic interview trap: access token vs ID token

DO NOT SWAP THEM

An access token is presented to the API. An ID token is intended for the OIDC client and carries authentication information. Do not use an ID token as a generic API access token.

Why choose JWT access tokens?

- Resource servers can validate tokens locally, which can reduce authorization-server lookups.
- Claims can carry context needed by APIs, such as subject, audience, scope, or authorization details.

Why not always choose JWT?

- Claims can become stale until expiry if permissions change.
- Immediate revocation is harder unless you add short expirations, deny-lists, key rotation, introspection, or another stateful mechanism.
- JWTs can leak readable claims if developers mistake encoding for encryption.
- Large JWTs add bandwidth and header overhead.

05 2026 security notes interviewers like

You do not need to recite RFC numbers. You do need the current instincts.

Use Authorization Code + PKCE

PKCE binds the authorization request to the code exchange. Public clients must use it under the OAuth Security BCP, and it is recommended for confidential clients too.

Avoid the implicit grant

Current OAuth security guidance says clients should generally avoid flows that return access tokens directly in the authorization response.

Never use the password grant

The Resource Owner Password Credentials grant must not be used under RFC 9700.

Validate redirect URIs strictly

Loose or wildcard redirect matching can turn an authorization flow into a token/code exfiltration path.

Validate JWT algorithms

Do not accept whatever alg the token asks for. Restrict acceptable algorithms and keys.

Validate iss + aud + exp

A cryptographically valid token can still be invalid for your application or API.

Use TLS

Bearer tokens are credentials. Whoever possesses one can often use it, unless sender-constrained mechanisms are involved.

Minimize scope and lifetime

Grant only the permissions and duration the client needs.

STATUS CHECK: OAUTH 2.1

As of September 2026, OAuth 2.1 is still an IETF Internet-Draft, not a final RFC. In an interview, you can say that modern OAuth practice already follows many of the consolidated OAuth 2.1 ideas, but cite OAuth 2.0 plus current Best Current Practices as the stable standards baseline.

Browser apps: an extra 2026 note

RFC 10017, published in August 2026, is the Best Current Practice for browser-based OAuth applications. That makes browser token handling, backend-for-frontend patterns, and exposure of bearer tokens especially relevant for frontend/system-design interviews.

06 Follow-up questions and model answers

Q Is JWT more secure than an opaque token?

A Not inherently. Security depends on validation, storage, transport, expiry, key management, audience restrictions, and the surrounding protocol. JWT mainly changes representation and validation tradeoffs.

Q Can a JWT be revoked?

A JWT itself does not provide a universal revocation mechanism. Common approaches are short expirations, token deny-lists, key rotation, session/grant revocation, or checking state/introspection when the design requires immediate control.

Q Can I put a password or secret inside a JWT payload?

A Do not assume the payload is secret. A normal signed JWT is readable by anyone who obtains it. Only put information there that is appropriate for the token's exposure model, or use encryption when the protocol actually calls for it.

Q What is the difference between authentication and authorization?

A Authentication answers "who are you?" Authorization answers "what are you allowed to do?" OAuth focuses on authorization. OIDC uses OAuth 2.0 mechanisms to add authentication and identity information.

Q What is PKCE?

A Proof Key for Code Exchange. The client creates a secret verifier and sends a derived challenge in the authorization request. The authorization code can only be redeemed by a party that later proves possession of the verifier.

Q Why is audience validation important?

A It stops a token minted for one API or relying party from being accepted by another unintended recipient.

Q What is the difference between roles and scopes?

A Scopes usually describe what a client is allowed to request/do through the token. Roles usually describe a subject's application-level position or entitlements. Exact semantics are application-specific.

07 Last-minute cheat sheet

OAuth	Authorization framework for delegated access.
JWT	Compact claims representation / token format.
OAuth token format	Not fixed. Access tokens can be opaque or structured.
OIDC	Authentication/identity layer built on OAuth 2.0.
Access token	For the resource server / API.
ID token	For the OIDC client, not a generic API credential.
Refresh token	For getting new access tokens from the authorization server.
JWT signature	Integrity/authenticity, not confidentiality.
JWT payload	Base64url-encoded, usually readable unless encrypted.
Must validate	Signature/algorithm + issuer + audience + expiry + intended token type/use.
Modern flow	Authorization Code + PKCE.
Avoid	Implicit grant and Resource Owner Password Credentials grant.

A clean closing answer

INTERVIEW CLOSE

So the short version is: OAuth defines the authorization flow and permissions. JWT defines one way of representing claims in a token. OAuth can issue JWT access tokens, but the two concepts are independent, and for login/identity I would bring OpenID Connect into the conversation.

Self-test

- Explain OAuth vs JWT in 20 seconds without saying “authentication.”
- Draw Authorization Code + PKCE from memory.
- Explain why a signed JWT is not automatically confidential.
- Explain access token vs ID token to a frontend engineer.
- Name three checks you perform when validating a JWT.
- Explain one advantage and one drawback of JWT access tokens versus opaque tokens.

Standards & current references

[RFC 6749 - The OAuth 2.0 Authorization Framework](#)

[RFC 7519 - JSON Web Token \(JWT\)](#)

[RFC 8225 - JSON Web Token Best Current Practices](#)

[RFC 9068 - JWT Profile for OAuth 2.0 Access Tokens](#)

[RFC 9700 - Best Current Practice for OAuth 2.0 Security](#)

[RFC 10017 - OAuth 2.0 for Browser-Based Applications \(Aug 2026\)](#)

[OAuth 2.1 Working Draft - draft-ietf-oauth-v2-1-15](#)